Assignment 1
Due by 11:59 pm on 27 September 2025

Covered contents:
Lectures 1 - 3

Submission method:
For questions 1 and 3, you should upload the source files to Canvas. For question 2, submission will be recorded once you have initiated the pull request.

If your have any questions, please email phycsan@ust.hk.

Question 1 [20 points]

Write a shell script titled '`organize.sh`' that automatically organizes files in a directory based on their extensions and provides a summary report.

Requirements:

Your script should

1. Accept a target directory as a command line argument.
   - If no argument is given, default to using the current directory.
   - You should also validate that the provided path exists and is indeed a directory (not file!). The script should output an error message in the event an invalid path is given.
2. Create organized subdirectories based on file types. If there are no certain file types, do not create the corresponding subdirectory.
   - `Images/` for .jpg, .jpeg, .png, .gif, .bmp
   - `Audio/` for .mp3, .wav
   - `Videos/` for .mp4, .wmv, .mov, .mvi
   - `Documents/` for .pdf, .doc, .docx, .txt, .md
   - `Scripts/` for .sh, .py, .js, .php
   - `Others/` for any other file extensions
3. Move files (not copy) into their appropriate subdirectories.
   - Only move regular files, **ignore directories and hidden files**
4. Output a report to the standard output (the terminal screen) that displays
   - Total number of files processed
   - Number of files moved to each category

To test out your script, you may generate various file types in a temporary directory, and execute the script.

Question 2 [10 points]

1. Fork the repo from `https://github.com/ycsan-ust/MSDM5001_Lecture2`
2. Clone the forked repo to your local machine
3. In "`student_names.txt`", type in your name on a new line
4. Make a commit
5. Push changes to you forked repo, and submit a pull request to me

Question 3 [20 points]

Download the Chinook sample database from here if you haven't already.

Create an SQL script titled `artist_revenue_analysis.sql` that performs the following task:

Find all artists who are in the top 20% by total revenue generated from their tracks. For each of these top-performing artists, calculate their revenue rank percentile, total revenue, number of albums, average revenue per album, and the percentage of their total revenue that comes from their single best-performing album. Additionally, show how many other artists (not just the top 20%) they outperformed in terms of total revenue. Order by total revenue in descending order.

Your output should look something like this:

Artist	**Percentile**	**Revenue**	**AlbumCount**	**Avg/Album**	**BestPercent**	**Outperformed**
...	...	...	...	...	...	...

Note: An artist being in the p^{th} percentile in revenue means that their revenue is greater than or equal to revenues of $p\%$ of artists.